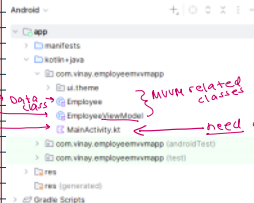
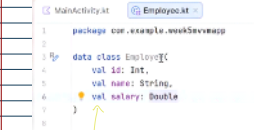


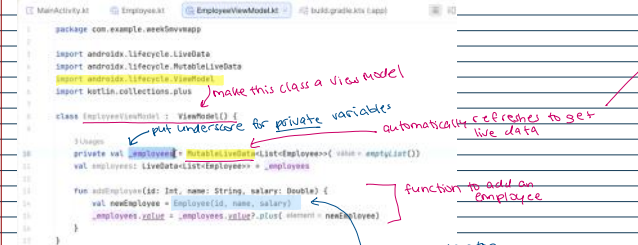
1) Employee MVVM App - Basic



Creating the Employee data class:



Create the ViewModel - determines the functions you will perform with the associated data class



can add additional functions to manipulate your data class

add() {} delete() {} update() {}

2) The MainActivity class

The advantages of using LiveData.

Using LiveData provides the following advantages:

Ensures your UI matches your data state

LiveData follows the observer pattern. LiveData notifies `Observer` objects when underlying data changes. You can consolidate your code to update the UI in these `Observer` objects. That way, you don't need to update the UI every time the app data changes because the observer does it for you.

No memory leaks

Observers are bound to `Lifecycle` objects and clean up after themselves when their associated lifecycle is destroyed.

No crashes due to stopped activities

If the observer's lifecycle is inactive, such as in the case of an activity in the back stack, then it doesn't receive any LiveData events.

No more manual lifecycle handling

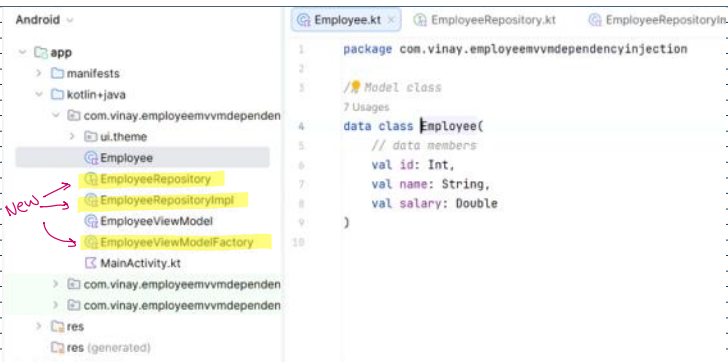
UI components just observe relevant data and don't stop or resume observation. LiveData automatically manages all of this since it's aware of the relevant lifecycle status changes while observing.

Always up to date data

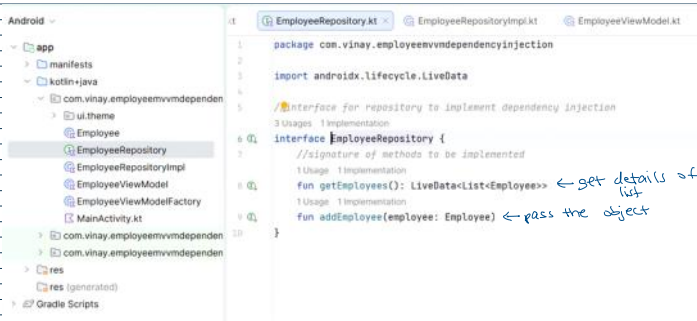
If a lifecycle becomes inactive, it receives the latest data upon becoming active again. For example, an activity that was in the background receives the latest data right after it returns to the foreground.

Proper configuration changes

If an activity or fragment is recreated due to a configuration change, like device rotation, it immediately receives the latest available data.

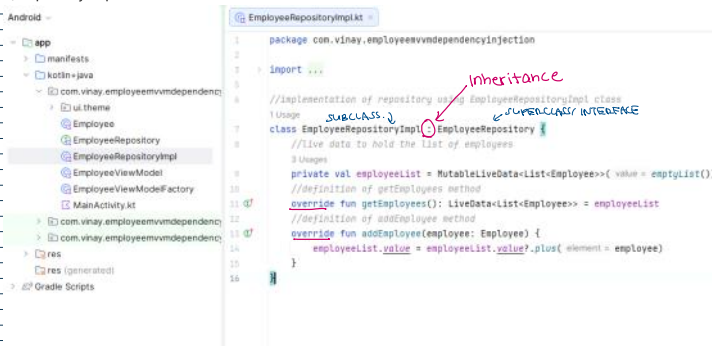


new repository - Implement dependency injection (vs in previous example, used a variable to pass the object)

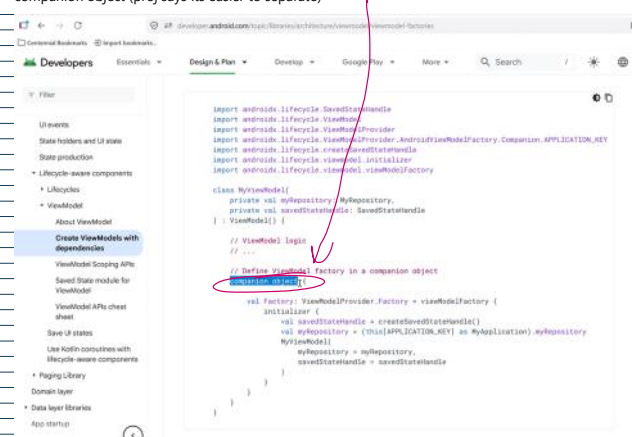


Repository
- Appropriate approach for implementing dependency injection

2) Repository Implementation



If you don't separate into View Model & ViewModelFactory, you will need to use a companion object (proj says its easier to separate)



3) View Model Factory



4) Main Activity

File Explorer: The project structure shows the following hierarchy: `app` (manifests, kotlin+java, data, di, ui.theme, viewmodel, views, ChapterFiveApplication, MainActivity), `com.example.architectureexample` (android, test), `res` (generated), and `Gradle Scripts`.

Package List: A list of packages including `com.example.architectureexample`, `data`, `di`, `ui.theme`, `viewmodel`, and `PetsRepositoryImpl`.

Code Snippets:

- `Pet.kt`: `data class Pet { val id: Int, val name: String, val species: String }`
- `PetsRepository.kt`: `interface PetsRepository { fun getPets(): List<Pet> }`
- `PetsRepositoryImpl.kt`: `class PetsRepositoryImpl: PetsRepository { override fun getPets(): List<Pet> { ... }`

Handwritten Notes:

- `Pet` is labeled as a **data class**.
- `PetsRepository` is labeled as an **interface**.
- `PetsRepositoryImpl` is labeled as an **implementation interface**.
- A note points to the `PetsRepositoryImpl` package, stating: **how to implement dependency injection**.

Pet.kt:

```

package com.example.architectureexample.data

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity
data class Pet {
    @PrimaryKey
    val id: Int
    val name: String
    val species: String
}

```

PetsRepository.kt:

```

package com.example.architectureexample.data

import com.example.architectureexample.data.Pet

interface PetsRepository {
    fun getPets(): List<Pet>
}

```

PetsRepositoryImpl.kt:

```

package com.example.architectureexample.data

import com.example.architectureexample.data.Pet

class PetsRepositoryImpl: PetsRepository {
    override fun getPets(): List<Pet> {
        return listOf(
            Pet(1, "Bella", "Dog"),
            Pet(2, "Luna", "Cat"),
            Pet(3, "Charlie", "Dog"),
            Pet(4, "Luna", "Cat"),
            Pet(5, "Casper", "Dog"),
            Pet(6, "Max", "Cat"),
            Pet(7, "Bailey", "Dog"),
            Pet(8, "Bailey", "Cat"),
            Pet(9, "Sadie", "Dog"),
            Pet(10, "Lily", "Cat")
        )
    }
}

```

PetsViewModel.kt:

```

package com.example.architectureexample.viewmodel

import androidx.lifecycle.ViewModel
import com.example.architectureexample.data.PetsRepository

class PetsViewModel(
    private val petsRepository: PetsRepository
): ViewModel() {
    fun getPets() = petsRepository.getPets()
}

```

PetList.kt:

```

package com.example.architectureexample.views

import androidx.recyclerview.widget.RecyclerView
import com.example.architectureexample.viewmodel.PetsViewModel
import com.example.architectureexample.data.Pet

class PetListAdapter(
    private val viewModel: PetsViewModel
): RecyclerView.Adapter<PetListAdapter.ViewHolder>() {
    fun PetListAdapter.ViewHolder {
        modifier = Modifier
        padding(16.dp)
        horizontalArrangement = Arrangement.spacedBy(16.dp)
    }
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val view = LayoutInflater.from(parent.context).inflate(R.layout.pet_item, parent, false)
        return ViewHolder(view)
    }
    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val pet = viewModel.getPets()[position]
        holder.modifier = Modifier
        holder.itemView.setOnClickListener {
            // TODO: Implement click listener
        }
    }
    override fun getItemCount(): Int {
        return viewModel.getPets().size
    }
}

```

W5 - Advanced Architecture and Development Page 6